

Limit Order Protocol Permit2Proxy Audit



February 10, 2026

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Privileged Roles	6
Medium Severity	7
M-01 ZKsync Era Has Different Official Permit2 Address	7
Low Severity	7
L-01 Missing Docstrings	7
L-02 Permit2Proxy Breaks Multi-Transaction Partial Fills due to Single-Use Permit2 Nonces	8
Notes & Additional Information	8
N-01 Missing Zero Address Check	8
N-02 Missing Security Contact	9
Conclusion	10
Appendix	11
Issue Classification	11

Summary

Type	DeFi	Total Issues	5 (3 resolved)
Timeline	From 2026-02-05 To 2026-02-05	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	2 (2 resolved)
		Notes & Additional Information	2 (0 resolved)

Scope

OpenZeppelin performed an audit of the [1inch/limit-order-protocol](#) repository at commit [a6148e1](#) that merges [pull request #400](#).

In scope were the following files:

```
contracts
├── extensions
│   ├── ImmutableOwner.sol
│   └── Permit2Proxy.sol
└── interfaces
    └── IPermit2TransferFrom.sol
```

System Overview

[Pull request #400](#) introduces a new [Permit2Proxy extension contract](#) used by the Limit Order Protocol, providing an alternative to [Permit2WitnessProxy contract](#) for a simpler Permit2 integration without witness data. The [Permit2Proxy](#) contract acts as an intermediary between 1inch's [AggregationRouter](#) and the [Permit2 contract](#) within the Limit Order Protocol to support [Permit2](#)-based signature transfers of maker and taker assets. It exposes one special function, [func_nZHTch](#), which intentionally matches the selector of the ERC-20 [transferFrom](#) function (`0x23b872dd`).

During deployment, the [AggregationRouter](#) is set as an immutable owner of the [Permit2Proxy](#) contract, ensuring that [func_nZHTch](#) can only be invoked through the [AggregationRouter](#). When an order is being filled, the [_callTransferFromWithSuffix](#) function calls [transferFrom](#) on [Permit2Proxy](#) (based on the maker/taker traits), which is intentionally set as the asset in the order data. This call includes additional suffix data containing the signature required for the [Permit2 call](#). This [transferFrom](#) call is passed to [Permit2: permitTransferFrom](#) which verifies the signature and calls [transferFrom](#) on the actual asset provided in the suffix data.

Security Model and Trust Assumptions

Throughout the in-scope codebase, the following trust assumptions were identified:

- The [Permit2Proxy](#) contract has an immutable ownership. The deployer of the contract is trusted to set correct ownership during deployment.
- Makers and takers are trusted to create accurate orders with appropriate suffix data in order to utilize this contract.
- Token owners (makers/takers) are expected to grant allowance to the Permit2 contract beforehand.

Privileged Roles

`Permit2Proxy` has an immutable owner set during deployment. Only this owner can call the `func_nZHTch` function.

Medium Severity

M-01 ZKsync Era Has Different Official Permit2 Address

The `_PERMIT2` address is set in the `Permit2Proxy` contract as `0x00000000000022D473030F116dDEE9F6B43aC78BA3`. The `Permit2Proxy` is expected to be deployed on every chain that supports `Permit2` contract. However, [according to official documentation](#), Zksync Era has a different Permit2 address.

Consider resolving this issue by checking the `chainId` in the constructor and setting the `Permit2` address to the officially supported address for the given chain. Alternatively, consider implementing `_getPermit2Address` functionality similar to 1inch's `solidity-utils` repository.

Update: Resolved in [pull request #413](#).

Low Severity

L-01 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `ImmutableOwner.sol`, the `IMMUTABLE_OWNER` state variable
- In `Permit2Proxy.sol`, the `Permit2Proxy` contract
- In `Permit2Proxy.sol`, the `func_nZHTch` function
- In `IPermit2TransferFrom.sol`, the `IPermit2TransferFrom` interface
- In `IPermit2TransferFrom.sol`, the `permitTransferFrom` function

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #413](#).

L-02 Permit2Proxy Breaks Multi-Transaction Partial Fills due to Single-Use Permit2 Nonces

When using `Permit2Proxy`, the maker embeds the `Permit2` permit parameters (including the nonce) and signature into the order's `makerAssetSuffix` at order creation time, and the transfer is executed via Permit2 `SignatureTransfer.permitTransferFrom`, which consumes the nonce on first use (via `_useUnorderedNonce`). As a result, if the order permits partial fills (i.e., the maker did not set `NO_PARTIAL_FILLS_FLAG`), the first partial fill succeeds but any subsequent fill attempt (to complete the remaining amount) will fail because the same permit nonce cannot be reused.

Makers can unintentionally create orders that appear to support partial fills at the protocol level but become unfillable after the first fill when routed through `Permit2Proxy`, leading to degraded UX, and potential lost execution opportunities. This can further cause unexpected reverts and operational issues for integrators and users relying on partial-fill behavior.

Consider documenting that `Permit2Proxy` orders should only be used for non-partial-fill orders and explicitly note that Permit2 `SignatureTransfer` permits are single-use because the nonce is consumed on first execution.

Update: Resolved in [pull request #413](#).

Notes & Additional Information

N-01 Missing Zero Address Check

When operations with the `address` parameters are performed, it is crucial to ensure that a zero address is not set. Setting an address to zero is problematic because it has special burn/renounce semantics. This action should be handled by a separate function to prevent accidental loss of access during value or ownership transfers.

Within `ImmutableOwner.sol`, the `_immutableOwner` operation is missing a zero-address check.

Consider adding a zero-address check before assigning a state variable.

Update: Acknowledged, not resolved.

N-02 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Throughout the codebase, multiple instances of contracts not having a security contact were identified:

- The [ImmutableOwner contract](#)
- The [Permit2Proxy contract](#)
- The [IPermit2TransferFrom interface](#)

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the [@custom:security-contact](#) convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: Acknowledged, not resolved.

Conclusion

The audited changes introduce the [Permit2Proxy](#) contract, enabling the integration of Uniswap Labs' [Permit2](#) with the limit order protocol, primarily enabling makers and takers to authorize token transfers via signatures instead of on-chain approvals.

During the review, one medium-severity issue pertaining to compatibility for ZKsync Era chain was identified. Several low-severity findings and informational notes aimed at enhancing code clarity and robustness were also reported.

The 1inch team is commended for their robust support and high responsiveness throughout the audit period.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.